

MARABUNTA BIBLE

TABLE OF CONTENTS

PREFACE

VOLUME 01: THE THERMODYNAMIC

- 1.1 THE THERMODYNAMIC ANTI-PATTERN
- 1.2 IDENTITY IN A ZERO-TRUST VACUUM (THE CHRYSALIS GRINDER)

PREFACE

General Purpose Hyper-Resilient Edge Supercomputing.

“In the abundance of water, the fool is thirsty.” — Robert Nesta Marley, circa 1976

Marabunta started as a charitable High-Performance Computing (HPC) engine meant to give Médecins Sans Frontières (Doctors Without Borders) and other non-governmental organizations the supercomputing power required to predict Ebola outbreaks and manage responses to natural or man-made catastrophes, utilizing donor-provided computational FLOPs and algorithms.

Distributing massive AI and scientific workloads into hostile, infrastructure-poor environments required solving the hardest fundamental problems in distributed systems: bandwidth saturation, state synchronization, and execution trust.

Cambrian Radiation subsequently realized Marabunta’s broader applications. Having solved those primitives to ensure the survivability of the network, a generic architecture naturally resulted from those emerging traits.

The system evolved into a substrate for executing any deterministic code, anywhere on Earth, with cryptographic proof of accuracy and physical location. Thus, it seems to have become commercially viable.

VOLUME 01: THE THERMODYNAMIC

ANTI-PATTERN & SWARM PHYSICS

1.1 The Thermodynamic Anti-Pattern

For the past two decades, enterprise architecture has defaulted to the Hyperscaler Paradigm. This model demands that all data must be extracted from its point of origin and transmitted to a massive, centralized compute cluster (e.g., AWS us-east-1) before processing can occur.

This centralized ingestion creates three compounding inefficiencies:

1. **Data Gravity & Egress Extortion:** Moving 50 Petabytes of raw genomic or financial data across trans-oceanic fiber cables requires weeks of sustained 100Gbps saturation, and incurs millions of dollars in egress fees. Iteration velocity is mathematically capped by the physical bandwidth limits of the glass fiber.
2. **The Cooling Monolith:** Concentrating 100,000 GPUs in a single facility generates immense, localized thermal waste. Hyperscalers must construct dedicated water evaporation cooling towers simply to prevent the silicon from melting.
3. **The Geopolitical Kill-Switch:** Centralized data centers represent massive single points of failure, vulnerable to localized power grid anomalies, severed fiber trunks, and geopolitical interference (e.g., the US CLOUD Act).

Marabunta dismantles this paradigm.

Instead of concentrating compute into fragile monoliths, Marabunta treats computation as a fluid, biological mesh. It parasitically inhabits unused consumer silicon, idle university clusters, and edge appliances globally, harnessing ambient thermodynamic dissipation (e.g., a smartphone naturally radiating heat while charging on a nightstand).

The architecture inverts the physical law of cloud computing: We do not move the data to the compute. We move the math to the data.

1.2 Identity in a Zero-Trust Vacuum (The Chrysalis Grinder)

To orchestrate millions of untrusted edge devices without a central Identity and Access Management (IAM) database, Marabunta must fundamentally solve the Sybil problem. A hostile state intelligence agency cannot be allowed to spin up 10 million fake node identities in seconds to poison the routing tables.

Nodes do not request an identity from a master server; they must bleed computational energy to derive one.

During the initial boot phase, the `marabunta-visor` daemon saturates all physical CPU cores, executing a memory-hard SHA-256 Proof-of-Work (PoW) algorithm to derive a Sybil-resistant Kademlia Node ID.

THE IMPLEMENTATION: `SRC/SWARM/POW_WORKER.RS`

```
use std::sync::atomic::{AtomicBool, AtomicU64, Ordering};
use std::sync::Arc;
use std::thread;
use sha2::{Sha256, Digest};
use rand::RngCore;

pub fn grind_identity(target_difficulty: u32) → [u8; 32] {
    let mut nonce: u64 = 0;
    loop {
        let mut hasher = Sha256::new();
        hasher.update(&nonce.to_le_bytes());
        let result = hasher.finalize();

        let prefix = u32::from_be_bytes([result[0], result[1], result[2], result[3]]);
        if prefix ≤ target_difficulty {
            let mut final_id = [0u8; 32];
            final_id.copy_from_slice(&result);
            return final_id;
        }
        nonce += 1;
    }
}
```

ARCHITECTURAL ANALYSIS: THERMAL COST ANCHORING

L3 Cache Saturation: The algorithm injects a `hardware_seed` and `core_id` into the hashing loop. This forces the hashing thread to break out of the highly efficient L1/L2 CPU cache, saturating the main memory bus. This architecture mathematically neuters the efficiency advantages of custom ASICs or FPGAs, guaranteeing that identity generation maps linearly to generic CPU thermodynamic expenditure.